

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok**
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A set (*halmaz*) adattípus

- A *halmazok* egyedi elemek rendezetlen összességei.
- A `set` *mutable* típus, de csak *immutable* elemeket tartalmazhat.

```
>>> primes = {2,7,5,2,3,5}
>>> primes.add(3)
>>> primes
{2, 3, 5, 7}
>>> primes.add(11)
>>> primes
{2, 3, 5, 7, 11}
>>> primes.add(('Péntek', 13))
>>> primes
{2, 3, 5, 7, 11, ('Péntek', 13)}
>>> primes.add(['list'])
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

A set (*halmaz*) adattípus műveletei

Operátorok

- egyoperandusú műveletek

`len()` a halmaz elemszáma

- kétoperandusú műveletek

| unió

& metszet

- különbség

^ szimmetrikus különbség

`in` tartalmazás

`not in` nem tartalmazás

```
len({-1, 1, 'egy', -1, "egy"})
```

```
{1, 2} | {2, 3}
```

```
{1, 2} & {2, 3}
```

```
{1, 2} - {2, 3}
```

```
{1, 2} ^ {2, 3}
```

```
1 in {'egy', 2}
```

```
3 not in set(range(5,10))
```

A set (*halmaz*) adattípus műveletei

Relációs műveletek

- kétoperandusú műveletek

`==` egyenlő

`!=` nem egyenlő

`<` (szigorúan) részhalmaz

`>` (szigorúan) szuperhalmaz

`<=` részhalmaz

`>=` szuperhalmaz

```
{1, 1, 2} == {1, 2, 2}
{3, 2, 1} != {1, 2, 3}
{1, 2}     <  {1, 2}
{1, 2, 3}  >  {1, 4}
{1, 2}     <= {1, 3, 4}
{1, 2, 3}  >= {3, 2, 1}
```

Konverzió set (*halmaz*) típusra

- `set(val)`

- Sztring: a sztring egyedi karaktereit sorban tartalmazó halmaz az eredmény
- List, Tuple: a lista/tuple egyedi elemeit tartalmazó halmaz az eredmény
- Dictionary: a dictionary kulcsait tartalmazó halmaz az eredmény
- Generátor típus: a generátor által visszaadott egyedi elemeket tartalmazó halmaz az eredmény

```
>>> set("Hello")
'e', 'l', 'o', 'H'
>>> set((3,2,1,2,3))
1, 2, 3
>>> set([1,2,3,2,1])
1, 2, 3
>>> set({1: 'egy', 2: 'kettő'})
1, 2
>>> set(range(3))
0, 1, 2
```

A set (*halmaz*) adattípus műveletei

Hasznos műveletek (módosítás)

- műveletek halmazelemekkel
 - `.add()` elem beszúrása
 - `.pop()` elem kivétele
 - `.remove()` elem törlése
 - `.discard()` elem törlése
- értékmódosító halmazműveletek
 - `.update()` elemek beszúrása
 - `.intersection_update()` elemek megtartása
 - `.difference_update()` elemek kivétele
 - `.symmetric_difference_update()` elemek „cseréje”
 - `.clear()` összes elem törlése

```
s.add(42)
s.pop()
s.remove(42)
s.discard(3)

s.update({2, 1, "négy"})
s.intersection_update({2, 1, "négy"})
s.difference_update({2, 1, "négy"})
s.symmetric_difference_update({2, 1})
s.clear()
```

A set (*halmaz*) adattípus műveletei

Hasznos műveletek (ellenőrzések)

- halmazok viszonya
 - `.isdisjoint()` diszjunktak?
 - `.issubset()` részhalmaz?
 - `.issuperset()` szuperhalmaz?
- halmazműveletek
 - `.union()` unió
 - `.intersection()` metszet
 - `.difference()` különbség
 - `.symmetric_difference()` szimmetrikus különbség
 - `.copy()` halmaz másolata

```
s.isdisjoint({3, "egy"})
s.subset({3, "egy"})
s.superset({3, "egy"})

s.union({2, 1, "négy"})
s.intersection({2, 1, "négy"})
s.difference({2, 1, "négy"})
s.symmetric_difference({2, 1, "öt"})
s2 = s.copy()
```

A set (*halmaz*) adattípus műveletei

Hasznos műveletek (aggregálás)

- függvények

`all()` mind igaz
`any()` van közte igaz
`max()` legnagyobb érték
`min()` legkisebb érték
`sum()` elemek összege

```
all({True, False, True})  
any({True, False, True})  
max({5, 3, 2, 4, 1})  
min({5, 3, 2, 4, 1})  
sum({5, 3, 2, 4, 1})
```



A frozenset (*halmaz*) adattípus

- A *halmazok* egyedi elemek rendezetlen összességei.
- A `frozenset` *immutable* típus, és csak *immutable* elemeket tartalmazhat.
- A `frozenset` típus tulajdonképpen a `set` típus *immutable* verziója.
- A `frozenset` ennek megfelelően nem rendelkezik a `set` olyan műveleteivel, amelyek megváltoztathatnák az értékét.

```
>>> primes = frozenset({2,7,5,2,3,5})
>>> primes.add(3)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
AttributeError: 'frozenset' object has no attribute 'add'
```

A frozenset (*halmaz*) adattípus műveletei

Hasznos műveletek

- halmazok viszonya

- `.isdisjoint()` diszjunktak?

- `.issubset()` részhalmaza?

- `.issuperset()` szuperhalmaza?

- halmazműveletek

- `.union()` unió

- `.intersection()` metszet

- `.difference()` különbség

- `.symmetric_difference()`

- szimmetrikus különbség

- `.copy()` halmaz másolata

- függvények

- `all()` mind igaz

- `any()` van közte igaz

- `max()` legnagyobb érték

- `min()` legkisebb érték

- `sum()` elemek összege

A set és frozenset (*halmaz*) adattípusok műveleteinek tulajdonságai

- A `set` *mutable* típus. Egyes műveletek a halmaz értékét változtatják meg!
- A `frozenset` *immutable* típus. Egyes műveletek új halmazt adnak vissza!

```
>>> a = b = {1,2,3}
>>> a is b
True
>>> b |= {4,5}
>>> a is b
True
>>> a
{1, 2, 3, 4, 5}
>>> b
{1, 2, 3, 4, 5}
```

```
>>> a = b = frozenset({1,2,3})
>>> a is b
True
>>> b |= {4,5}
>>> a is b
False
>>> a
frozenset({1, 2, 3})
>>> b
frozenset({1, 2, 3, 4, 5})
```

A set és frozenset (*halmaz*) adattípusok bejárása

Elemek szerint

- A halmaz elemeit a `for` utasítás segítségével be tudjuk járni
 - A halmazelemeket elvileg véletlenszerű sorrendben kapjuk meg

```
>>> s = {"Egy", 2, (3, 4), 5}:
>>>
>>> for e in s:
...     print(f"{e!r}")
...
'Egy'
2
5
(3, 4)
```

```
>>> s = ["Egy", 2, (3, 4), 5]:
>>> s = frozenset(s):
>>> for e in s:
...     print(f"{e!r}")
...
2
5
(3, 4)
'Egy'
```

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - **Tér legtávolabbi pontjai**
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tér legtvolabbi pontjai [1/2]

ter.py (v1.0) [1-24]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8A háromdimenziós tér pontjai közül keressük meg a két legtvolabbit!
9"""
10from math import sqrt
11
12def beolvas():
13    try:
14        p = input("x,y,z? ")
15        p = p.split(",")
16        p = (float(p[0]), float(p[1]), float(p[2]))
17        return p
18    except:
19        return None
20
21def tav(p, q):
22    (px, py, pz) = p
23    (qx, qy, qz) = q
24    return sqrt((px-qx)**2 + (py-qy)**2 + (pz-qz)**2)
```

Tér legtavolabbi pontjai [2/2]

ter.py (v1.0) [26–49]

```
26 def legtavolabb(pontok):
27     maxtav = -1.0
28     p = None
29     q = None
30     while pontok:
31         i = pontok.pop()
32         for j in pontok:
33             if (ujtav := tav(i, j)) > maxtav:
34                 maxtav = ujtav
35                 p = i
36                 q = j
37     return p, q
38
39 def main():
40     pontok = set()
41     while p := beolvas():
42         pontok.add(p)
43     p, q = legtavolabb(pontok)
44     print("A két legtavolabbi pont:")
45     print(p)
46     print(q)
47
48 if __name__ == "__main__":
49     main()
```

Tér legtávolabbi pontjai

A probléma

- Adott a térben valahány pont, melyik a két legtávolabbi?



Tér legtovábbi pontjai

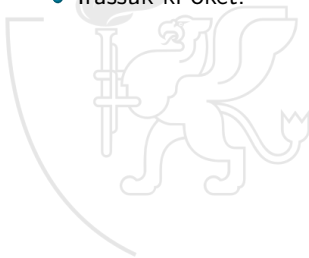
ter.py (v0.1) [1–14]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8A háromdimenziós tér pontjai közül keressük meg a két legtovábbi!
9"""
10def main():
11    pass
12
13if __name__ == "__main__":
14    main()
```

Tér legtávolabbi pontjai

Főprogram

- Kezdjük magas szinten:
 - Kezdetben nincsenek pontjaink.
 - Amíg vannak, olvassunk be pontokat (egy-egy koordináta-hármaszt).
 - A pontok tárolására használjunk halmazt!
 - Ha ugyanazokat a koordinátákat adjuk is meg, a pont egyszer fog szerepelni a halmazban.
 - A pontok közül válasszuk ki az egymástól legtávolabbiakat.
 - Írassuk ki őket.



Tér legtávolabbi pontjai

ter.py (v0.2) [1–26]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8A háromdimenziós tér pontjai közül keressük meg a két legtávolabbit!
9"""
10def beolvas():
11    return None
12
13def legtavolabb(pontok):
14    return None, None
15
16def main():
17    pontok = set()
18    while p := beolvas():
19        pontok.add(p)
20    p, q = legtavolabb(pontok)
21    print("A két legtávolabbi pont:")
22    print(p)
23    print(q)
24
25if __name__ == "__main__":
26    main()
```

Tér legtávolabbi pontjai

Pont beolvasása

- Készítsük el a pontok beolvasását végző függvényt.



A try-except utasítás

- Ha valamilyen lehetséges hiba következményeit szeretnénk kezelni.
 - Mit csináljunk, ha futási hiba lép fel egy utasítás végrehajtása során?
 - Ha a felhasználó nem számot ad meg, akkor nem számolunk.
 - De ha nullát, akkor is baj van.

```
try:
    v = int(input('Kérek egy egész számot: '))
    if v % 2 == 0:
        print('A szám páros, a reciproka %f!' % (1 / v))
    else:
        print('A szám páratlan!')
except:
    print('Valami hiba történt!')
```

Tér legtavolabbi pontjai

ter.py (v0.3) [10–32]

```
10 def beolvas():
11     try:
12         p = input("x,y,z? ")
13         p = p.split(",")
14         p = (float(p[0]), float(p[1]), float(p[2]))
15         return p
16     except:
17         return None
18
19 def legtavolabb(pontok):
20     return None, None
21
22 def main():
23     pontok = set()
24     while p := beolvas():
25         pontok.add(p)
26     p, q = legtavolabb(pontok)
27     print("A két legtavolabbi pont:")
28     print(p)
29     print(q)
30
31 if __name__ == "__main__":
32     main()
```

Tér legtávolabbi pontjai

Fejlesztés

- A legtávolabbi pontok meghatározásához minden pontot minden ponttal össze kell hasonlítani ...



Tér legtvolabbi pontjai

ter.py (v0.4) [19–38]

```
19 def tav(p, q):
20     return 0.0
21
22 def legtvolabb(pontok):
23     for i in pontok:
24         for j in pontok:
25             ujtav = tav(i, j)
26     return None, None
27
28 def main():
29     pontok = set()
30     while p := beolvas():
31         pontok.add(p)
32     p, q = legtvolabb(pontok)
33     print("A két legtvolabbi pont:")
34     print(p)
35     print(q)
36
37 if __name__ == "__main__":
38     main()
```

Tér legtávolabbi pontjai

Fejlesztés

- A legtávolabbi pontok meghatározásához minden pontot minden ponttal össze kell hasonlítani ...
- ... miközben kiválasztjuk az összehasonlítások közül a legnagyobb távolságot (és a hozzá tartozó két pontot).



Tér legtávolabbi pontjai

ter.py (v0.5) [19–44]

```
19 def tav(p, q):
20     return 0.0
21
22 def legtavolabb(pontok):
23     maxtav = -1.0
24     p = None
25     q = None
26     for i in pontok:
27         for j in pontok:
28             if (ujtav := tav(i, j)) > maxtav:
29                 maxtav = ujtav
30                 p = i
31                 q = j
32     return p, q
33
34 def main():
35     pontok = set()
36     while p := beolvas():
37         pontok.add(p)
38     p, q = legtavolabb(pontok)
39     print("A két legtávolabbi pont:")
40     print(p)
41     print(q)
42
43 if __name__ == "__main__":
44     main()
```

Tér legtávolabbi pontjai

Ponttávolság

- Szükség lesz két pont távolságára is.



Tér legtvávolabbi pontjai

ter.py (v0.6) [1-24]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_előadás_példaprogram
4SZTE_TTIK,_Informatikai_Intézet,_Szoftverfejlesztés_Tanszék
5Gergely_Tamás,_gertom@inf.u-szeged.hu,_2024-05-31
6cc-by-nc-nd
7
8A_háromdimenziós_tér_pontjai_közül_keressük_meg_a_két_legtvávolabbt!
9"""
10from math import sqrt
11
12def beolvas():
13    try:
14        p = input("x,y,z? ")
15        p = p.split(",")
16        p = (float(p[0]), float(p[1]), float(p[2]))
17        return p
18    except:
19        return None
20
21def tav(p, q):
22    (px, py, pz) = p
23    (qx, qy, qz) = q
24    return sqrt((px-qx)**2 + (py-qy)**2 + (pz-qz)**2)
```

Tér legtávolabbi pontjai

Gondolkodjunk még

- Ha p és q távolsága d , akkor mennyi lesz q és p távolsága?
 - Felesleges két pont távolságát oda-vissza összehasonlítani, mert (az Euklideszi térben) ugyanaz.
 - Ha egy pontot már minden másik ponttal összehasonlítottunk, akkor akár ki is dobhatjuk a halmazból.
 - Valójában, ha már kiválasztottuk a pontot amivel az összes többit össze fogjuk hasonlítani, azonnal ki is vehetjük a halmazból (így saját magával sem kell majd összehasonlítani).
 - Ha később is kell az összes pont, akkor ez nem jó ötlet, de most nem lesz rájuk szükségünk.



Tér legtvolabbi pontjai [1/2]

ter.py (v1.0) [1-24]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8A háromdimenziós tér pontjai közül keressük meg a két legtvolabbit!
9"""
10from math import sqrt
11
12def beolvas():
13    try:
14        p = input("x,y,z? ")
15        p = p.split(",")
16        p = (float(p[0]), float(p[1]), float(p[2]))
17        return p
18    except:
19        return None
20
21def tav(p, q):
22    (px, py, pz) = p
23    (qx, qy, qz) = q
24    return sqrt((px-qx)**2 + (py-qy)**2 + (pz-qz)**2)
```

Tér legtávolabbi pontjai [2/2]

ter.py (v1.0) [26–49]

```
26 def legtavolabb(pontok):
27     maxtav = -1.0
28     p = None
29     q = None
30     while pontok:
31         i = pontok.pop()
32         for j in pontok:
33             if (ujtav := tav(i, j)) > maxtav:
34                 maxtav = ujtav
35                 p = i
36                 q = j
37     return p, q
38
39 def main():
40     pontok = set()
41     while p := beolvas():
42         pontok.add(p)
43     p, q = legtavolabb(pontok)
44     print("A két legtávolabbi pont:")
45     print(p)
46     print(q)
47
48 if __name__ == "__main__":
49     main()
```